1. Introduction and Overview
2. Focus Search System
3. Google Shoot Your Shot Dating System
4. YouTube Video Search
5. Harmful Content Detection
6. Video Recommendation System
7. Event Recommendation System
8. Ad CTR Prediction Around Purchases
9. Similar Listings on Vacation Rental Marketplace
10. Personalized News Feed
11. People You May Know

05
Harmful Content Detection

Many social media platforms such as Facebook [1], LinkedIn [2], and Twitter [3] have standard guidelines to enforce integrity and keep their platforms safe for users. Some of the common harmful content includes pornography, violence, and crowd that are hurtful to the community. It is essential to have technologies and resources in place to effectively handle harmful content and bad actors. We can divide the problem into two broad categories, which are:

- Harmful content: Text, image, or video that violates platform policies.
- Bad actors: Users that accounts, users posting, organized unethical activities, and other unethical behaviors.

In this chapter, we focus on detecting harmful content such as harmful content posted in a profile post, we design a system that proactively monitors text, pictures, detects harmful content, and removes or demotes them if the content violates the platform's guidelines. To understand how comprehensive such harmful content detection system in practice, refer to [4], [5], [6].

Figure 1.1: Harmful content detection system

Clarifying Requirements

Here is a typical interaction between a candidate and an interviewer.

Candidate: Does the system detect harmful content in text and pictures?
Interviewer: Both are equally important. For simplicity, let's focus on detecting harmful content only.

Candidate: Do we need to consider the context of the post when detecting harmful content?
Interviewer: The context of a post can be text, image, video, or any combination.

Candidate: What languages are supported? Is it English only?
Interviewer: The system should detect harmful content in many languages. For simplicity, assume we can use a pre-trained multilingual model to extract the textual content.

Candidate: Do we need to detect if a harmful content post is being to identify? I can think of violence, hurtful, hate speech, misinformation, etc. Are there harmful categories we need to consider?
Interviewer: Yes, focus on the major ones. Misinformation is more complex and controversial. For simplicity, let's not focus on misinformation.

Candidate: Are there any human annotations available to label post manually?
Interviewer: Yes, there are some annotations. Note that the ground truth labeling humans to label all of them would be very expensive and time-consuming. Therefore, you can assume human annotation is available to label a point current use-by date. For example, 10,000 per day.

Candidate: Among users to report harmful content is beneficial for understanding where the system is failing. Can I assume this feature is available?
Interviewer: Great point. Yes, users can report harmful posts.

Candidate: Do we need to consider users' personal feed and timeline?
Interviewer: No. Explaining to users why we remove a post is essential. It helps users to ensure they align their future posts with the guidelines.

Candidate: What is the system's latency requirement? Do we need a real-time prediction, i.e., the system detects harmful content as a post is being created, or is it acceptable if the system detects harmful content a few minutes after post creation?
Interviewer: Time is a very important question. What are your thoughts?

Candidate: In my opinion, the requirements for effective harmful content might vary. For example, violent content may require real-time detection, while other types might tolerate delays.
Interviewer: Those are fair assumptions.

So, let's summarize the problem statement. We will design a harmful content detection system, which identifies harmful posts, from text, videos, or pictures and informs the user why the post violates community guidelines. It aims to support harmful posts, or any combination of these, and the content can be in different languages. Users can report harmful posts.

Frame the Problem as an ML Task

Defining the ML objective

We define our ML objective as accurately classifying harmful posts. The reason is that if we can accurately detect harmful posts, we can remove or demote them quickly to users platform.

Specifying the system's input and output

The system receives a post as input, and it outputs the probability that the post is harmful.

Figure 1.2: Input/output of the system

To make accurate predictions, the system should consider all modalities, can't discuss two commonly used training methods to handle multimodal problems: Early fusion and late fusion.

Late fusion

With late fusion, the model combines different modalities independently, then combine their predictions to make a final prediction. The figure below illustrates how late fusion works.

Figure 1.3: Late fusion

The advantage of late fusion is that we can use separate, pre-trained state-of-the-art models for each modality. Since fusion is a single model in here, we only need to collect training data for that model. Second, the model considers all the modalities, so it can handle a lacking or noisy modality. For example, if a post contains only text, the model can potentially make a prediction. However, learning this task is more difficult as the model due to the complex relationships between modalities. In the absence of sufficient training data, it's challenging for the model to learn complex relationships between modalities.

Which fusion method should we use?

Late fusion is a good choice if we have sufficient data or a capacity point that may be harmful cannot, even if each modality is large enough. Additionally, late fusion works well if posts being published every day, the model has enough data to learn the task.

Choosing the right ML category

In this section, we examine the following ML category options:

- Supervised learning
- Unsupervised learning
- Multi-label classifier
- Multi-task classifier

Single-binary classifier

In this option, a single binary model predicts harmful posts. The model's output is the probability of the post being harmful (Figure 3.0). Since the output is a binary outcome, the model is a binary classifier.

Figure 1.4: Supervised learning

The shortcoming of this system is that it's difficult to determine which class of harmful, such as violence, a post belongs to. This is critical in cases to inform users.

- It is a good choice if we want only one label (user) at the system only outputs a binary value indicating whether the post is a phishing harmful or not. We have to consider what kind of harmful that belongs to.
- It is not easy to identify harmful classes in which the system doesn't perform well, meaning we cannot improve the system for underperforming classes.

Since it is essential to explain why a post is removed, a single binary classifier is not a good option.

One-binary classifier per class

In this option, we adapt one binary classifier for each harmful class. For example, if we have 5 classes, each model determines if a post belongs to that class or not. Since multiple models are used, we can train and evaluate the probability of the post being classified as a harmful class.

Figure 1.5: One-binary classifier per class

The advantage of this option is that we can easier to users why a post was taken down. In addition, we can monitor different models' performance and improve them separately.

- It is a good choice if we want to explain why a post was taken down. In addition, we can monitor different models' performance and improve them separately.
- It is not easy to identify harmful classes in which the system doesn't perform well, meaning we cannot improve the system for underperforming classes.

Since it is essential to explain why a post is removed, a single binary classifier is not a good option.

Multi-label classifier

In this option, we adapt one binary model to predict multiple harmful classes. For example, if we have 5 classes, each model determines if a post belongs to one or more of the 5 classes. Since multiple models are used, we can train and evaluate the probability of the post being classified as a harmful class.

Figure 1.6: Multi-label classifier

The advantage of this option is that we can easier to users why a post was taken down. In addition, we can monitor different models' performance and improve them separately.

- It is a good choice if we want to explain why a post was taken down. In addition, we can monitor different models' performance and improve them separately.
- It is not easy to identify harmful classes in which the system doesn't perform well, meaning we cannot improve the system for underperforming classes.

Since it is essential to explain why a post is removed, a single binary classifier is not a good option.

Multi-task classifier

Multi-task learning refers to the process in which a model learns multiple tasks simultaneously. This allows the model to learn correlations between tasks. By doing so, we avoid unnecessary computations when a certain layer transformation is beneficial for multiple tasks.

In our case, we need to determine several of the post, such as violence and nudity, as different tasks and use a multi-task classifier. The model's input can vary. As Figure 3.0 shows, each task requires a unique output layer, shared layers and task-specific layers.

Figure 1.7: Multi-task classifier

Shared layers: A shared layer, as shown in Figure 3.0, is a set of hidden layers that transform input features into new ones. These newly transformed features are used to make predictions for each of the harmful classes.

Figure 1.8: Shared layers

Task-specific layers: Task-specific layers are a set of independent fit layers, unlike called classification heads. Each classification head transforms features in a way that is optimal for predicting a specific harmful probability.

Figure 1.9: Task-specific layers

Multi-task classification has three advantages. First, it is not expensive to train and maintain since we use a single model. Second, the shared layers transform the features in a way that is beneficial for each task. This prevents redundant computations and makes training more efficient. Finally, the shared layers learn a set of features that can contribute to the learning of other tasks. This is especially helpful when limited data is available for a particular task.

Because of these advantages, we employ a multi-task classification method. Figure 3.0 shows how our frame the problem.

Figure 3.0: Frame the problem as an ML task

Data Preparation

Data engineering

We have the following data available:

- Text
- Image
- User-post interactions

Users

A user data schema is shown below.

id	username	Age	Gender	City	Country	Email

Table 5.1: User data schema

Posts

Posts data consists fields such as author, time of upload, etc. Table 5.2 shows some of the most important attributes. In practicy, we have much more attributes associated with each post.

Post id	Author id	Text content	Timestamp	Textual content location	Image or video	Links
1	1	I LOVE CHICAGO	1653612345	Today, I am starting my new job.	http://cdn.content.com/1.jpg	
2	1	BEAUTIFUL WEEK!	1653612567	The video showed our Paris trip	http://cdn.content.com/2.mp4	gofamous.com/2ctu7qm1
3	4	BEAUTIFUL WEEK!	1653612567	What is a good recipe for Thanksgiving?	http://cdn.content.com/3.jpg	

Table 5.2: Post data

User-post interactions

User-post interactions data primarily contain users' reactions to posts, such as likes, comments, saves, shares, etc. Users can also report a post as harmful. Table 5.3 shows what the data schema looks like for these interactions.

User ID	Post ID	Interaction type	Interaction value	Timestamp
11	3	Impression		1624567890
4	23	Like		1624567267
11	7	Comment	This is disgusting	1624567890
11	9	Share		1624567456
11	7	Report	violence	1624567890

Table 5.3: User-post interaction data

Feature engineering

In the "Frame the problem as an ML task" section, we framed the problem as a multi-task classification task where the input is the post, and the output is a vector of probabilities that can be added a predefined threshold to pass. A post might contains the following details:

- Textual content
- Image or video
- Text embedded in the post
- Audio
- Contextual information

Let's look at each element.

Textual content

The textual content of a post can be used to determine whether a post is harmful or not. As described in Chapter 6 YouTube Video Search, text data is usually prepared in two steps:

- **Text normalization:** Cleans and standardizes text.
- **Tokenization:** Breaks down the normalized text into individual tokens, which can be a word, a character, or a subword. In this chapter, I'll exercise the text and subword feature sections, remember a NN-based methods can be used. Statistical methods like TF-IDF and BM25 are not suitable for this task because they don't capture the semantic meaning of the text, which is crucial for understanding the intent behind a post. For our system, consider the semantics of the textual content is important for detecting harmful, as we want the NN-based method. To convert text into a feature vector, we use a pre-trained Transformer-based language model such as BERT, which has been fine-tuned on social media data.

Pre-training the tokenizer from scratch is the best approach for training the core of the model. Because this is a rare process, using 3rd party pre-trained tokenizer is not ideal.

- BERT uses a tokenizer of vocab size 30k. Thus, it does not produce meaningful embeddings for tests in other languages.

DistilBERT [7], is a more efficient variant of BERT, addresses these two issues. It has vocabulary size of 119k, which covers 104 languages from various Wikipedia articles. Although it's not perfect, it has an advantage to learn more about multilingual language models, refer to [8].

Image or video

You can enrich the textual content of a post by looking at the images or video within it. As described in Chapter 6 YouTube Video Search, text data is usually prepared in two steps:

- **Preprocessing:** resizing, cropping, and normalizing the data.
- **Feature extraction:** after preprocessing, we use a pre-trained model to convert pre-processed data into a feature vector. This process is crucial for capturing the essential information in the visual data, which is then used for further analysis or classification tasks. Popular models for this include CNN architectures like VGG, ResNet, and EfficientNet. For videos, you frames sampled from the video (1-2 FPS might suffice).

User interactions in the post

It is also possible to determine whether a post is harmful based on user reactions, especially when the content is ambiguous. As shown in Figure 6.23, with more comments, it becomes increasingly evident the post contains content related to self-harm.

Figure 6.23: Post with self-harm comments

Since user reactions are crucial in determining harmful content, let's examine some of the features we can engineer based on user interactions.

The number of likes, shares, comments, and reports: We usually scale these numerical values to speed up convergence during model training.

Comments: As demonstrated in Figure 6.23, comments can help us identify harmful content. For feature preparation, we convert comments into numerical representations by doing the following:

- Use the same pre-trained model we employed earlier to obtain the embedding of each comment.
- Aggregate (e.g. average the embeddings) to obtain a fixed embedding.

A summary of the features we have described so far can be found in Figure 6.24.

Concatenated features
Transformer Encoder
Transformer Encoder
Transformer Encoder

- **Time of day:** This is the time of day when the author made a post. We bucketize this into multiple categories, such as morning, noon, afternoon, evening or night. We use one-hot encoding to represent this feature.
- **Device:** The device the author uses, such as a smartphone or desktop computer. One-hot encoding is used to represent this feature.

Figure 5.15 summarizes some of the most important features of the harmful content detection system.

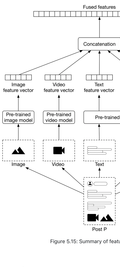

Figure 5.15: Summary of feature engineering

Model Development

Model selection

A neural network is the most common model used for multi-task learning. In developing our model, we employ neural networks.

When choosing a neural network, what factors should be considered? It is necessary to determine the neural network's architectural design and the optimal hyperparameter selection, such as its hidden layers, activation function, learning rate, etc. The optimal choice of hyperparameters is usually determined by hyperparameter tuning. Let's briefly go over it.

Hyperparameter tuning is the process of finding the best values for hyperparameters in order to produce the best performance for a model. To tune hyperparameters, grid search is commonly used. The procedure involves training a new model for each combination of hyperparameter values, evaluating each model, then selecting the hyperparameters that lead to the best model. If you are interested in learning more about hyperparameter tuning, refer to [14].

Model training

Constructing the dataset

To train the multi-task classification model, we first need to construct the dataset. The dataset comprises model inputs (features) and outputs (labels) that the model is expected to predict. To construct inputs, we process posts offline in batches and compute fused features as described earlier. These features can be stored in a feature store for future training. In order to create labels for each input, we have two options:

- Hand labeling
- Natural labeling

With hand labeling, human contractors label posts manually. This option produces accurate labels, but it is expensive and time-consuming. With natural labeling, we rely on user reports to label posts automatically. While this option results in noisier labels, labels are produced more quickly. For the evaluation dataset, we use hand labeling to prioritize the accuracy of labels, and for the training dataset, we use natural labeling to prioritize labeling speed. A data point from the constructed dataset is shown in Figure 5.16.

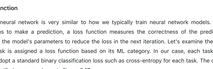

Figure 5.16: A constructed data point

Choosing the loss function

Training a multi-task neural network is very similar to how we typically train neural network models. A forward propagation performs computations to make a prediction, a loss function measures the correctness of the prediction, and a backward propagation optimizes the model's parameters to reduce the loss in the next iteration. Let's examine the loss function. In multi-task training, each task is assigned a loss function based on its ML category. In our case, each task is framed as a binary classification, so we adopt a standard binary classification loss such as cross-entropy for each task. The overall loss is computed by combining task-specific losses, as shown in Figure 5.17.

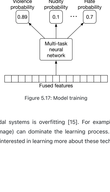

Figure 5.17: Model training

A common challenge in training multimodal systems is overfitting [15]. For example, when the learning speed varies across different modalities, one modality (e.g., image) can dominate the learning process. Two techniques to address this issue are gradient blending and focal loss. If you are interested in learning more about these techniques, refer to [16] [17].

Evaluation

Offline metrics

To evaluate the performance of a binary classification model, offline metrics such as precision, recall, and F1 score are commonly used. However, precision or recall alone is not sufficient to understand the overall performance. For example, a model with high precision might have a very low recall. The precision-recall (PR) curve and receiver operating characteristic (ROC) curve address those limitations. Let's explore each one.

PR-curve. PR curve shows the trade-off between precision and recall of the model. As Figure 5.18 shows, we obtain a PR curve by plotting the precision of the model using different probability thresholds, ranging from 0 to 1 . To summarize the trade-offs between precision and recall, PR-AUC (the area under the precision-recall curve) calculates the area beneath the PR curve. In general, a high PR-AUC indicates a more accurate model.

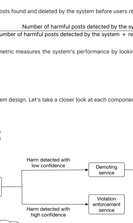

Figure 5.18: PR curve

ROC curve. The ROC curve shows the trade-offs between the true positive rate (recall) and the false positive rate. Similar to the PR curve, ROC-AUC summarizes the model's performance by calculating the area under the ROC curve.

ROC and PR curves are two different ways to summarize the performance of a classification model. To learn about the differences between the PR curve and the ROC curve, read [18].

In our case, we use both ROC-AUC and PR-AUC as our offline metrics.

Online metrics

Let's explore a few important metrics to capture how safe the platform is.

Prevalence. This metric measures the ratio of harmful posts which we didn't prevent and all posts on the platform.

$$\text{Prevalence} = \frac{\text{Number of harmful posts we didn't prevent}}{\text{Total number of posts on the platform}}$$

The shortcoming of this metric is that it treats harmful posts equally. For example, one harmful post with 100 K views or impressions is more harmful than two posts with 10 views each.

Harmful impressions. We prefer this metric over prevalence. The reason is that the number of harmful posts on the platform does not show how many people were affected by those posts, whereas the number of harmful impressions does capture this information.

Valid appeals. Percentage of posts that were deemed harmful, but appealed and reversed.

$$\text{Appeals} = \frac{\text{Number of reversed appeals}}{\text{Number of harmful posts detected by the system}}$$

Proactive rate. Percentage of harmful posts found and deleted by the system before users report it.

$$\text{Proactive rate} = \frac{\text{Number of harmful posts detected by the system}}{\text{Number of harmful posts detected by the system} \ + \ \text{reported by users}}$$

User reports per harmful class. This metric measures the system's performance by looking into user reports for each harmful class.

Serving

Figure 5.19 shows the high-level ML system design. Let's take a closer look at each component.

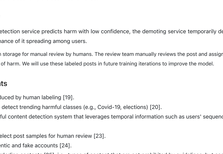

Figure 5.19: ML system design

Harmful content detection service

Given a new post, this service predicts the probability of harm. According to the requirements, some types of harm should be handled immediately due to their sensitivity. When this happens, the violation enforcement service removes the post immediately.

Violation enforcement service

The violation enforcement service immediately takes down a post if the harmful content detection service predicts harm with high confidence. It also notifies the user why the post was removed.

Demoting service

If the harmful content detection service predicts harm with low confidence, the demoting service temporarily demotes the post in order to decrease the chance of it spreading among users.

Then, the post is stored in storage for manual review by humans. The review team manually reviews the post and assigns a label from one of the predefined classes of harm. We will use these labeled posts in future training iterations to improve the model.

Other Talking Points

- Handle biases introduced by human labeling [19].
- Adapt the system to detect trending harmful classes (e.g., Covid-19, elections) [20].
- How to build a harmful content detection system that leverages temporal information such as users' sequence of actions [21] [22].
- How to effectively select post samples for human review [23].
- How to detect authentic and fake accounts [24].
- How to deal with borderline contents [25], i.e., types of content that are not prohibited by guidelines, but come close to the red lines drawn by those policies.
- How to make the harmful content detection system efficient, so we can deploy it on-device [26].
- How to substitute Transformer-based architectures with linear Transformers to create a more efficient system [27] [28].

References

1. Facebook's inauthentic behavior. https://transparency.fb.com/policies/community-standards/inauthentic-behavior/.
2. LinkedIn's professional community policies. https://www.linkedin.com/legal/professional-community-policies.
3. Twitter's civic integrity policy. https://help.twitter.com/en/rules-and-policies/election-integrity-policy.
4. Facebook's integrity survey. https://arxiv.org/abs/2009.10311.pdf.
5. Pinterest's violation detection system. https://medium.com/pinterest-engineering/how-pinterest-fights-misinformation-hate-speech-and-self-harm-content-with-machine-learning-195fb73b40ef.
6. Abusive detection at LinkedIn. https://engineering.linkedin.com/blog/2019/spotlight-fighting-abusive-forest.
7. WPIE method. https://ai.facebook.com/blog/community-standards-report/.
8. BERT paper. https://arxiv.org/pdf/1810.04805.pdf.
9. Multilingual DistilBERT. https://huggingface.co/distilbert-base-multilingual-cased.
10. Multilingual language models. https://arxiv.org/pdf/2107.00676.pdf.
11. CLIP model. https://openai.com/blog/clip/.
12. SimCLR paper. https://arxiv.org/pdf/2002.05709.pdf.
13. VideoMoCo paper. https://arxiv.org/pdf/2103.05905.pdf.
14. Hyperparameter tuning. https://cloud.google.com/ai-platform/training/docs/hyperparameter-tuning-overview.
15. Overfitting. https://en.wikipedia.org/wiki/Overfitting.
16. Focal loss. https://amaarora.github.io/posts/2020-06-29-FocalLoss.html.
17. Gradient blending in multimodal systems. https://arxiv.org/pdf/1905.12681.pdf.
18. ROC curve vs precision-recall curve. https://machinelearningmastery.com/roc-curves-and-precision-recall-curves-for-classification-in-python/.
19. Introduced bias by human labeling. https://labeyourdata.com/articles/bias-in-machine-learning.
20. Facebook's approach to quickly tackling trending harmful content. https://ai.facebook.com/blog/harmful-content-can-evolve-quickly-our-new-ai-system-adapts-to-tackle-it/.
21. Facebook's TIES approach. https://arxiv.org/pdf/2002.07917.pdf.
22. Temporal interaction embedding. https://www.facebook.com/atscaleevents/videos/2396853072323/.
23. Building and scaling human review system. https://www.facebook.com/atscaleevents/videos/2201519863329695/.
24. Abusive account detection framework. https://www.youtube.com/watch?v=YeX4MdUOJ8k.
25. Borderline contents. https://transparency.fb.com/features/approach-to-ranking/content-distribution-guidelines/content-borderline-to-the-community-standards/.
26. Efficient harmful content detection. https://about.fb.com/news/2021/12/metas-new-ai-system-tackles-harmful-content/.
27. Linear Transformer paper. https://arxiv.org/pdf/2006.04768.pdf.
28. Efficient AI models to detect hate speech. https://ai.facebook.com/blog/how-facebook-uses-super-efficient-ai-models-to-detect-hate-speech/.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
help@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.